

Agentes Inteligentes

Franz Mayr

`mayr@ort.edu.uy`

Universidad ORT Uruguay

31 de mayo de 2026

Aprendizaje y Planificación

Algoritmo DP para estimar π_* (Value Iteration)

Inicializar $V(s)$ arbitrariamente $\forall s \in \mathcal{S}$

Repetir:

$\Delta \leftarrow 0$

Para cada $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

hasta que $\Delta < \theta$ para algún umbral θ pequeño.

Algoritmo DP para estimar π_* (Value Iteration)

Inicializar $V(s)$ arbitrariamente $\forall s \in \mathcal{S}$

Repetir:

$\Delta \leftarrow 0$

Para cada $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

hasta que $\Delta < \theta$ para algún umbral θ pequeño.

Este método requiere un **modelo del ambiente**, $p(s', r | s, a)$. Por eso decimos que es un método *model-based*.

Algoritmo MC para estimar π_*

Inicializar:

$\pi \leftarrow$ política ε -soft arbitraria (ε -soft: $\pi(a | s) > 0 \ \forall s, a$)

$Q(s, a) \in \mathbb{R}$ arbitrariamente $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Retornos(s, a) \leftarrow$ lista vacía, $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

Generar un episodio según π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$.

$G \leftarrow 0$

Para cada paso del episodio, $t = T - 1, T - 2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Si (S_t, A_t) no aparece en $(S_0, A_0), (S_1, A_1), \dots, (S_{t-1}, A_{t-1})$:

Agregar G a $Retornos(S_t, A_t)$

$Q(S_t, A_t) \leftarrow$ promedio($Retornos(S_t, A_t)$)

$A^* \leftarrow \arg \max_a Q(S_t, a)$ (desempatando al azar)

$$\pi(a | S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon / |\mathcal{A}(S_t)| & \text{si } a = A^* \\ \varepsilon / |\mathcal{A}(S_t)| & \text{si } a \neq A^* \end{cases}$$

Algoritmo MC para estimar π_*

Inicializar:

$\pi \leftarrow$ política ε -soft arbitraria (ε -soft: $\pi(a | s) > 0 \ \forall s, a$)

$Q(s, a) \in \mathbb{R}$ arbitrariamente $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Retornos(s, a) \leftarrow$ lista vacía, $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

Generar un episodio según π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$.

$G \leftarrow 0$

Para cada paso del episodio, $t = T - 1, T - 2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Si (S_t, A_t) no aparece en $(S_0, A_0), (S_1, A_1), \dots, (S_{t-1}, A_{t-1})$:

Agregar G a $Retornos(S_t, A_t)$

$Q(S_t, A_t) \leftarrow$ promedio($Retornos(S_t, A_t)$)

$A^* \leftarrow \arg \max_a Q(S_t, a)$ (desempatando al azar)

$$\pi(a | S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon / |\mathcal{A}(S_t)| & \text{si } a = A^* \\ \varepsilon / |\mathcal{A}(S_t)| & \text{si } a \neq A^* \end{cases}$$

Este método **no** requiere un modelo del ambiente; se basa en **interacciones reales** con el ambiente. Es un método *model-free*.

OBSERVACIÓN: También puede usarse con interacciones **simuladas**.

Algoritmo TD para estimar π_* (Q-learning)

Inicializar $Q(s, a)$ arbitrariamente $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 hasta que S sea terminal

Algoritmo TD para estimar π_* (Q-learning)

Inicializar $Q(s, a)$ arbitrariamente $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 hasta que S sea terminal

Este método **no** requiere un modelo del ambiente; se basa en interacciones reales con el ambiente. Por eso, es un método *model-free*.

OBSERVACIÓN: También puede usarse con interacciones simuladas.

Modelo del ambiente

Un **modelo** es algo que el agente puede usar para predecir la forma en que el ambiente responderá a una acción.

Modelo del ambiente

Un **modelo** es algo que el agente puede usar para predecir la forma en que el ambiente responderá a una acción.

Dados un estado S y una acción A , un modelo devuelve una predicción de la recompensa R y el nuevo estado S' .

Cuando el modelo es estocástico, hay varias recompensas y nuevos estados posibles, cada uno con su probabilidad asociada.

Modelo del ambiente

Un **modelo** es algo que el agente puede usar para predecir la forma en que el ambiente responderá a una acción.

Dados un estado S y una acción A , un modelo devuelve una predicción de la recompensa R y el nuevo estado S' .

Cuando el modelo es estocástico, hay varias recompensas y nuevos estados posibles, cada uno con su probabilidad asociada.

Distinguimos dos tipos de modelos:

- ▶ **Modelo de distribución:** puede devolver cada una de las posibilidades y sus probabilidades.
- ▶ **Modelo de muestreo:** sólo devuelve una posibilidad, obtenida según las probabilidades.

EJEMPLO: Modelar la suma de 12 dados.

Modelo de distribución → prob. de cada tiro posible (difícil de computar)

Modelo de muestreo → un tiro posible por vez (sencillo de computar)

Modelo del ambiente

Método	Tipo	Requiere
Progr. Dinámica	<i>Model-based</i>	Modelo de distribución
MC / TD	<i>Model-based</i>	Modelo de muestreo
	<i>Model-free</i>	—

Aprendizaje y Planificación

Planificación (o *planning*): Proceso que, a partir de un modelo del ambiente (típicamente de distribución), construye una política para interactuar con el ambiente modelado. Ocurre íntegramente **offline**.

EJEMPLO: Los métodos DP son algoritmos de planificación.

Aprendizaje y Planificación

Planificación (o *planning*): Proceso que, a partir de un modelo del ambiente (típicamente de distribución), construye una política para interactuar con el ambiente modelado. Ocurre íntegramente **offline**.

EJEMPLO: Los métodos DP son algoritmos de planificación.

Aprendizaje: Proceso que construye una política en base a **experiencia real (online)**, generada al interactuar con el ambiente.

Aprendizaje y Planificación

Planificación (o *planning*): Proceso que, a partir de un modelo del ambiente (típicamente de distribución), construye una política para interactuar con el ambiente modelado. Ocurre íntegramente **offline**.

EJEMPLO: Los métodos DP son algoritmos de planificación.

Aprendizaje: Proceso que construye una política en base a **experiencia real (online)**, generada al interactuar con el ambiente.

Los métodos MC y TD fueron concebidos como métodos de aprendizaje, pero también se los puede convertir en métodos de planificación, al aplicarlos a **experiencia simulada** con un modelo (típicamente de muestreo).

Aprendizaje y Planificación

Planificación (o *planning*): Proceso que, a partir de un modelo del ambiente (típicamente de distribución), construye una política para interactuar con el ambiente modelado. Ocurre íntegramente **offline**.

EJEMPLO: Los métodos DP son algoritmos de planificación.

Aprendizaje: Proceso que construye una política en base a **experiencia real (online)**, generada al interactuar con el ambiente.

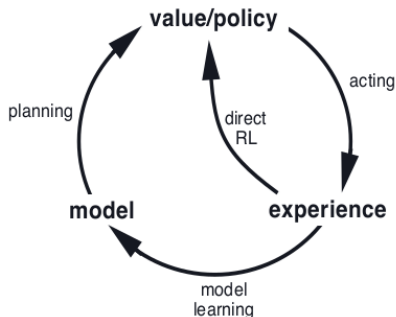
Los métodos MC y TD fueron concebidos como métodos de aprendizaje, pero también se los puede convertir en métodos de planificación, al aplicarlos a **experiencia simulada** con un modelo (típicamente de muestreo).

Ahora veremos cómo integrar planificación y aprendizaje en una técnica unificada, más general: **planificación online**.

Planificación online

Mediante la **planificación online**, un agente puede aprovechar de dos maneras la experiencia real de interactuar con el ambiente:

1. para mejorar el modelo (*model learning*),
2. para mejorar su estimación de la función de valor y su política, usando las técnicas ya vistas (*direct RL*).



Planificación online: Dyna-Q

Inicializar $Q(s, a)$

$\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S_{aux} \leftarrow S'$

$S \leftarrow S_{aux}$

 hasta que S sea terminal

Planificación online: Dyna-Q

Inicializar $Q(s, a)$ y *Modelo*(s, a) $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

(1) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

(2) *Modelo*(S, A) $\leftarrow R, S'$ (suponiendo un ambiente determinista)

$S_{aux} \leftarrow S'$

(3) Repetir n veces:

$S \leftarrow$ elegir al azar un estado ya observado

$A \leftarrow$ elegir al azar una acción ya tomada en S

$R, S' \leftarrow$ *Modelo*(S, A)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S_{aux}$

 hasta que S sea terminal

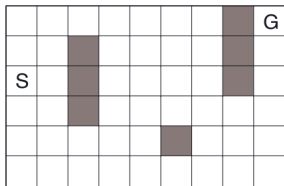
Pasos: (1) *direct RL*; (2) *model learning*; (3) *planning*.

Si omitimos el código en verde, el algoritmo que queda es Q-learning.

Ejemplo: Laberinto

Acciones: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ Tarea episódica con descuento $\gamma = 0,95$.

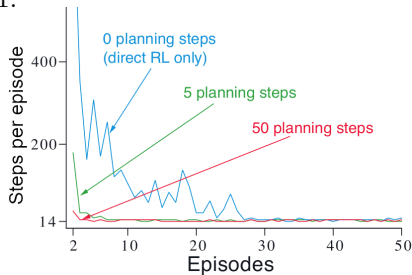
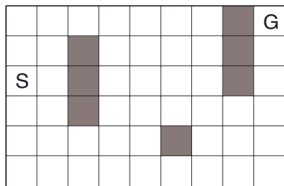
El agente empieza en S, la recompensa es 0 en todas las transiciones, excepto al llegar a G, donde es +1.



Ejemplo: Laberinto

Acciones: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ Tarea episódica con descuento $\gamma = 0,95$.

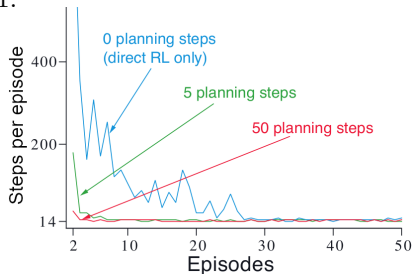
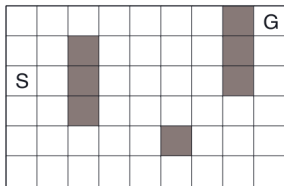
El agente empieza en S, la recompensa es 0 en todas las transiciones, excepto al llegar a G, donde es +1.



Ejemplo: Laberinto

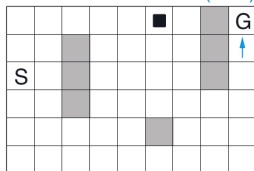
Acciones: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ Tarea episódica con descuento $\gamma = 0,95$.

El agente empieza en S, la recompensa es 0 en todas las transiciones, excepto al llegar a G, donde es +1.

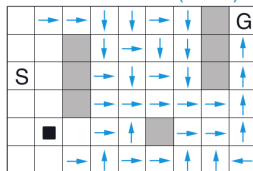


Políticas encontradas en la mitad del segundo episodio:

WITHOUT PLANNING ($n=0$)



WITH PLANNING ($n=50$)



Métodos directos e indirectos de RL

“Both direct and indirect methods have advantages and disadvantages. Indirect methods often make fuller use of a limited amount of experience and thus achieve a better policy with fewer environmental interactions. On the other hand, direct methods are much simpler and are not affected by biases in the design of the model. Some have argued that indirect methods are always superior to direct ones, while others have argued that direct methods are responsible for most human and animal learning. Related debates in psychology and artificial intelligence concern the relative importance of cognition as opposed to trial-and-error learning, and of deliberative planning as opposed to reactive decision making (see Chapter 14 for discussion of some of these issues from the perspective of psychology). Our view is that the contrast between the alternatives in all these debates has been exaggerated, that more insight can be gained by recognizing the similarities between these two sides than by opposing them. For example, in this book we have emphasized the deep similarities between dynamic programming and temporal-difference methods, even though one was designed for planning and the other for model-free learning.”

Sutton & Barto (2018), p.162

Revisitamos Dyna-Q

Inicializar $Q(s, a)$ y *Modelo*(s, a) $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

(1) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

(2) *Modelo*(S, A) $\leftarrow R, S'$ (suponiendo un ambiente determinista)

$S_{aux} \leftarrow S'$

(3) Repetir n veces:

$S \leftarrow$ elegir al azar un estado ya observado

$A \leftarrow$ elegir al azar una acción ya tomada en S

$R, S' \leftarrow$ *Modelo*(S, A)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S_{aux}$

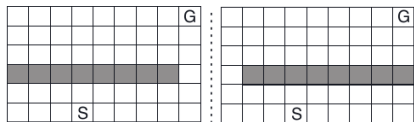
 hasta que S sea terminal

Pasos: (1) *direct RL*; (2) *model learning*; (3) *planning*.

Si omitimos el código en verde, el algoritmo que queda es Q-learning.

Dyna-Q+

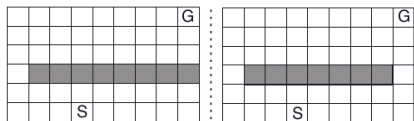
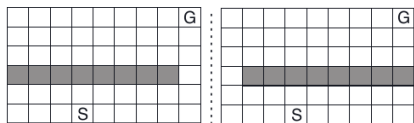
Ejemplos de **cambios en el ambiente**: (antes | después)



Dyna-Q demora un tiempo
en corregir la política.

Dyna-Q+

Ejemplos de **cambios en el ambiente**: (antes | después)

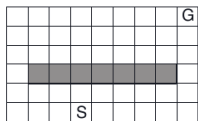
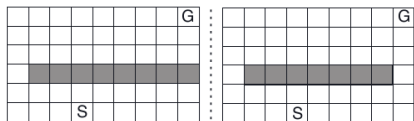
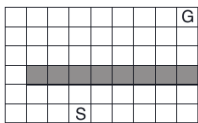
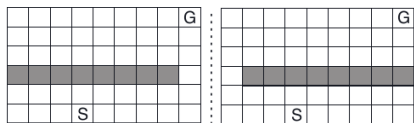


Dyna-Q demora un tiempo en corregir la política.

Aparecen oportunidades para mejorar la política, pero es improbable que Dyna-Q las encuentre.

Dyna-Q+

Ejemplos de **cambios en el ambiente**: (antes | después)



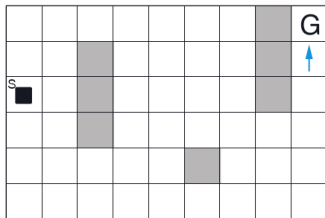
Dyna-Q demora un tiempo en corregir la política.

Aparecen oportunidades para mejorar la política, pero es improbable que Dyna-Q las encuentre.

Dyna-Q+ agrega un incentivo para volver a visitar pares (S, A) que hace tiempo no se visitan, para poder descubrir cambios en el ambiente y así mejorar la política.

Barrido priorizado (*prioritized sweeping*)

En el ciclo de planificación de Dyna-Q, el par (S, A) se elige al azar, lo cual puede ser muy ineficiente: ¿para qué actualizar los valores de estados de los que no tenemos información nueva?



Ejemplo: Al comenzar el segundo episodio, elegir (S, A) al azar será casi siempre inútil.

Barrido priorizado (*prioritized sweeping*)

Inicializar $Q(s, a)$, $Modelo(s, a) \forall s, a$; $ColaPr \leftarrow$ Cola de prioridad vacía

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

$Modelo(S, A) \leftarrow R, S'$

$P \leftarrow |R + \gamma \max_a Q(S', a) - Q(S, A)|$

 Si $P > \theta$, insertar (S, A) en $ColaPr$ con prioridad P

$S_{aux} \leftarrow S'$

 Repetir n veces, mientras $\neg vacía(ColaPr)$:

$S, A \leftarrow primero(ColaPr)$

$R, S' \leftarrow Modelo(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

 Repetir para cada $\bar{S}, \bar{A}$ que llevan a S (según $Modelo$):

$\bar{R} \leftarrow$ recompensa para $\bar{S}, \bar{A}, S$ (según $Modelo$)

$P \leftarrow |\bar{R} + \gamma \max_a Q(S, a) - Q(\bar{S}, \bar{A})|$

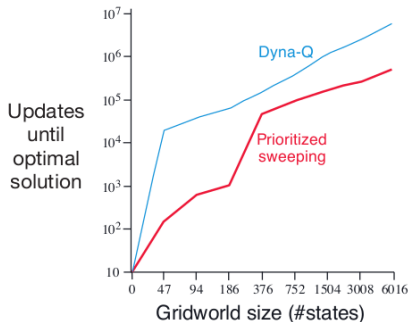
 Si $P > \theta$, insertar $(\bar{S}, \bar{A})$ en $ColaPr$ con prioridad P

$S \leftarrow S_{aux}$

hasta que S sea terminal

Barrido priorizado (*prioritized sweeping*)

Ejemplo: En tareas como el laberinto visto, la técnica de barrido priorizado mejora dramáticamente la velocidad respecto de Dyna-Q.



(OBSERVACIÓN: El eje vertical está en escala logarítmica.)

Resumen: Aprendizaje y Planificación

- ▶ Modelo del ambiente
- ▶ Modelo de distribución vs. modelo de muestreo
- ▶ Métodos *model-based* vs. *model-free*
- ▶ Aprendizaje vs. Planificación
- ▶ Planificación online: Dyna-Q y Dyna-Q+